

Universidade Federal do Rio Grande do Sul
Instituto de Informática

Relatório - Trabalho 1

INF01112 - Arquitetura e Organização de
Computadores II

Henrique Sangiovanni, Johann Hardt, Vítor Feijó

Professor: Arthur Lorenzon



Porto Alegre
31 de outubro de 2025

Sumário

1	Implementação da Cache	1
2	Implementação do LRU	2
3	Implementação do LFU	3
4	1° Caso: Vetor com alta localidade temporal	4
5	2° Caso: Vetor com alta localidade espacial	5
6	3° Caso: Caso híbrido	6

1 Implementação da Cache

Inicialmente, define-se os parâmetros da memória cache: tamanho e valores máximos e mínimos dos dados alocados.

```
constexpr int CACHE_SIZE = 10;  
constexpr int MAX_VALUE = 100;  
constexpr int MIN_VALUE = 0;
```

Figura 1: parâmetros da cache.

Cada endereço da cache é representada pela seguinte estrutura, contendo o dado e uma *flag* de validade:

```
struct CacheAddress {  
    int data = 0;  
    bool valid = false;  
};
```

Figura 2: *struct* dos endereços da memória

Para fins de simplicidade, implementou-se uma classe abstrada contendo métodos genéricos para todas as políticas de memória cache:

get_info: Imprime a quantidade total de *hits* e *misses* registrados na memória cache.

print_cache: Exibe o valor e a validade atual de todos os endereços de memória.

search_value: Verifica se um valor específico está presente na cache, retornando seu índice caso esteja e -1 caso não.

reset_cache: Limpa todos os índices da cache e zera os *hits* e *misses*, reiniciando seu estado.

2 Implementação do LRU

Na implementação da política de substituição LRU (Least Recently Used), utilizou-se uma lista dinâmica(`std::list`) de inteiros chamada `access_history` para armazenar o endereço dos valores acessados em ordem cronológica, sendo o primeiro item da lista o endereço mais recente.

```
list<int> access_history;
```

Em cada busca, primeiro é verificado se o valor já está na memória utilizando o método `search_value`. Se estiver, o endereço vai para o começo da lista e o número de *hits* é incrementado.

```
}else{
    access_history.remove(search_result); // updates history
    access_history.push_front(search_result);
    hits +=1;
```

Porém, se o valor não for encontrado, `search_value` retorna -1, incrementa-se o número de *hits* e o valor é alocado de acordo com a política LRU: primeiro se tenta colocar o dado em algum endereço vazio

```
int search_result = search_value(value);
if(search_result < 0){ //search returns -1 on miss
    misses += 1;
    //first tries to allocate in an empty address
    for(int i=0; i< CACHE_SIZE; i++){
        if (cache[i].valid == false){
            cache[i].valid = true;
            cache[i].data = value;
            access_history.push_front(i);
            return;
        }
    }
}
```

e se isso não for possível, coloca-se o dado no endereço mais antigo (último na lista), movendo-o para o começo para atualizar o registro de uso dos endereços.

```
int address = access_history.back();
cache[address].data = value;
access_history.remove(address); // updates history
access_history.push_front(address);
```

3 Implementação do LFU

A implementação da política LFU (Least Frequently Used) foi realizada por meio de um vetor estático de contadores, no qual cada posição representa a frequência de acesso do respectivo dado, sendo o índice do vetor o valor do dado e o conteúdo a sua frequência. Essa implementação foi adotada pela sua simplicidade e rapidez em números inteiros pequenos, por isso limitou-se o intervalo de valores possíveis nos parâmetros da memória cache.

```
int freq[MAX_VALUE - MIN_VALUE + 1] = {};
```

Independente do dado buscado, sua frequência deve ser incrementada

```
freq[value - MIN_VALUE]++;
```

Caso o dado já esteja na cache (novamente se utiliza *search_value*), o número de *hits* é incrementado.

```
} else  
    hits++;
```

Se o valor não estiver na cache é necessário aloca-lo no endereço do valor acessado meno frequentemente. Durante a substituição, o algoritmo compara as frequências de todos os valores presentes na cache e substitui aquele com a menor frequência de acesso pelo novo valor. A subpolítica de desempate adotada foi a escolha do valor com o menor endereço de memória.

```
if(val_index < 0) { //Returns -1 on miss. Must evict a variable or fill empty slot.  
    misses++;  
    int lowest_freq_val = cache[0].data; //Initialize variable to search for lowest frequency in cache.  
    int eviction_index = 0;  
    int i = -1;  
    do {  
        //Search for lowest frequency or empty address.  
        i++;  
        if (freq[lowest_freq_val] > freq[cache[i].data] || !cache[i].valid) {  
            eviction_index = i;  
            lowest_freq_val = cache[i].data; // Senseless if the address is empty, but inconsequential.  
        }  
    } while(cache[i].valid && i < CACHE_SIZE - 1);  
    cache[eviction_index].data = value;  
    cache[eviction_index].valid = true;
```

4 1° Caso: Vetor com alta localidade temporal

```
case 1:
  //Vector for LRU best case.
  entries = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
            0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
            10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
            10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
            10, 11, 12, 13, 14, 15, 16, 17, 18, 19};
```

O vetor apresenta uma sequência de valores repetida duas vezes, o que polui a memória com valores de alta frequência. Assim, quando uma nova sequência de valores diferentes aparece na política LFU troca todos os novos valores pelo da primeira posição da cache a cada alocação, até que os novos valores acumulem frequência suficiente para superar os anteriores. Por outro lado, esse padrão de acesso não prejudica a LRU, o que é comprovado pelos testes.

Final results:

Hits:	Misses:
LRU: 30	LRU: 20
LFU: 10	LFU: 40

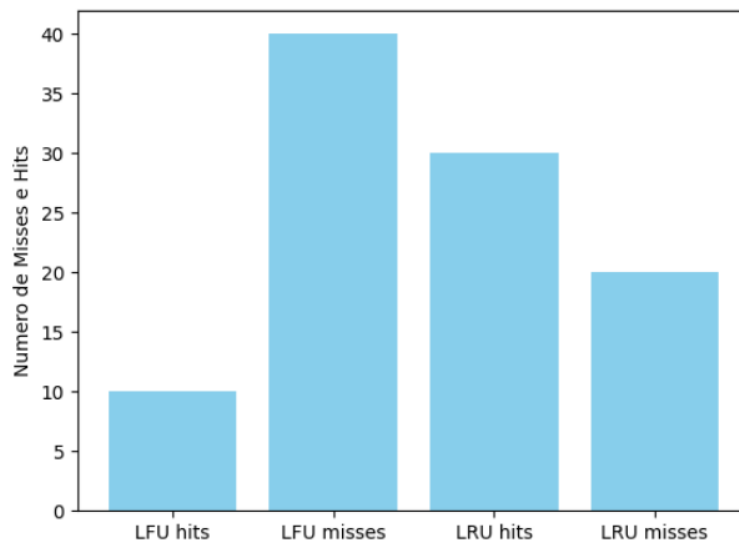


Gráfico feito com matplotlib no python

5 2° Caso: Vetor com alta localidade espacial

```
case 2:
//Vector for LFU best case.
entries = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
           0, 1, 2, 3, 4, 5,
           10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
           0, 1, 2, 3, 4, 5,
           10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
           0, 1, 2, 3, 4, 5, 6};
```

O vetor possui uma parte da sequência inicialmente carregada na memória de dados que é repetida entre duas outras sequências completamente diferentes. Esse padrão de acesso desfavorece a política LRU, pois não há uma forte localidade espacial: os mesmos dados estão distribuídos e espaçados na cache, sendo substituídos antes de serem reutilizados.

Contrariamente, a LFU é menos afetada com esse padrão de acesso, visto que é impactada majoritariamente pela localidade temporal (frequência alta de um subconjunto reduzido de valores).

Final results:

Hits:	Misses:
LRU: 01	LRU: 49
LFU: 18	LFU: 32

O favorecimento da LFU é nítido no gráfico:

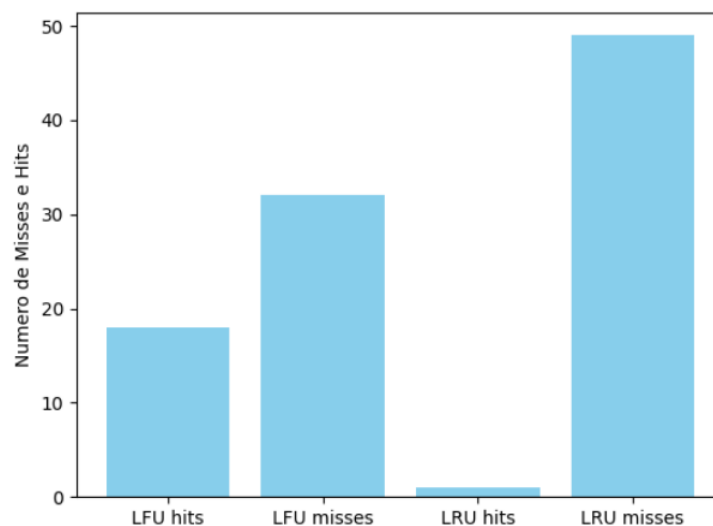


Gráfico feito com matplotlib no python

6 3° Caso: Caso híbrido

```
case 3:
//Vector with even result.
entries = //Fase A: Favorece LFU
{0, 1, 2, 3, 4,
10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
0, 1, 2, 3, 4,
10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
// Fase B: Favorece LRU
20, 21, 22, 23, 24, 25, 26, 27, 28, 29,
20, 21, 22, 23, 24, 25, 26, 27, 28, 29};
```

O vetor começa com uma mesma sequência de caracteres. No entanto, como essa sequência é muito grande, há pouca localidade espacial, o que desfavorece o desempenho da política LRU. Já na segunda parte do vetor, uma sequência menor é introduzida, criando a situação contrária, a qual é mais favorável para o LRU.

Para a LFU, entretanto, a repetição de caracteres na primeira parte gera alguns *hits*, mas a alta frequência dos valores iniciais na segunda parte provoca uma sequência de *misses*, quando novos valores passam a ser acessados.

Final results:

Hits:	Misses:
LRU: 10	LRU: 40
LFU: 09	LFU: 41

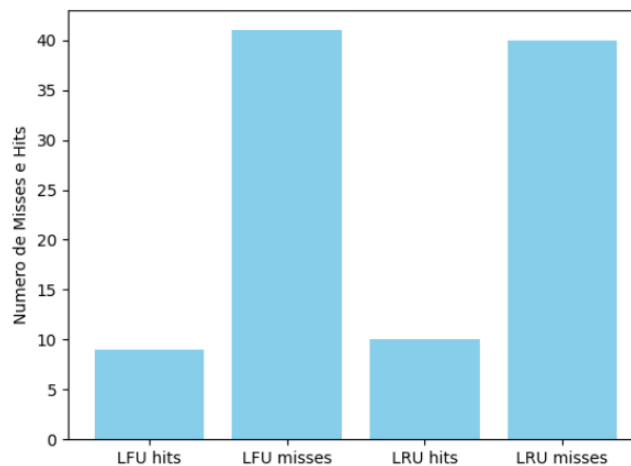


Gráfico feito com matplotlib no python